Resilient Autonomy: A Next-Generation Avionics Architecture for Self-Governing Launch Vehicles

1. Introduction: The Paradigm Shift to Onboard Autonomy

The exploration of space has historically been defined by a "ground-centric" operational paradigm. From the Apollo program to the early Space Shuttle era, the locus of decision-making resided firmly in Mission Control Centers (MCCs), where teams of flight controllers monitored telemetry, deliberated on anomalies, and uplinked command sequences for execution. This model, classified by the European Space Agency (ESA) as Autonomy Level E1 (execution under ground control), relies on the assumption of reliable, low-latency communication links and the availability of human cognition to resolve complex, unforeseen problems. However, the evolving landscape of spaceflight—characterized by reusable launch vehicles (RLVs) performing dynamic propulsive landings, deep-space robotic explorers venturing into communication-denied environments, and mega-constellations requiring automated traffic management—renders the E1 model obsolete.

We are currently witnessing a transition toward Autonomy Level E4: "Goal-Oriented Mission Execution." In this regime, the spacecraft is not merely an automated puppet executing a time-tagged script; it is an autonomous agent capable of perceiving its environment, reasoning about its internal health and external constraints, and generating its own plans to maximize mission utility. This shift is driven by three critical imperatives: the physics of latency, the economics of reusability, and the complexity of the orbital environment.

1.1 The Latency and Resilience Imperative

For missions to the outer solar system, round-trip light time delays can exceed several hours. The "terror" of the Entry, Descent, and Landing (EDL) phase on Mars, which lasts approximately seven minutes, occurs entirely within a communication blackout from the perspective of Earth-based operators. Consequently, a vehicle entering the Martian atmosphere must be capable of handling aerodynamic instabilities, trajectory dispersions, and potential subsystem failures without human intervention. Similarly, for RLVs returning to Earth, the timescales of dynamic pressure changes and aerodynamic instability during supersonic retro-propulsion are measured in milliseconds, far faster than any human-in-the-loop control loop could manage. The avionics must therefore possess "reflexes"—immediate, deterministic responses to safety-critical events—coupled with "cognition"—the ability to replan the mission if the primary landing site is obstructed or if main engine performance degrades.

1.2 The Architectural Challenge: reconciling Safety with Intelligence

The central engineering challenge in realizing E4 autonomy lies in the architectural tension

between *safety* and *intelligence*. Safety-critical avionics, governed by standards such as DO-178C (software) and DO-254 (hardware), rely on determinism, predictability, and rigorous verification. Conversely, modern artificial intelligence (AI) and machine learning (ML) algorithms—essential for tasks like computer vision-based landing or adaptive fuel optimization—are often stochastic, computationally intensive, and difficult to formally verify. Developing a "smart" rocket requires an avionics architecture that can host these two disparate worlds simultaneously. It necessitates a fault-tolerant operating system capable of strict Time and Space Partitioning (TSP) to prevent an experimental AI planner from starving the flight control loop of CPU cycles. It requires a communications backbone, such as Time-Triggered Ethernet (TTEthernet), that guarantees the delivery of stability augmentation signals while accommodating the bursty data traffic of autonomous agents. Furthermore, it demands a new class of "middleware" that allows legacy flight software frameworks like NASA's Core Flight System (cFS) to interoperate with modern robotics frameworks like Space ROS, enabling the rapid integration of advanced autonomy capabilities.

This report presents a comprehensive analysis of such an architecture. It details the integration of fault-tolerant real-time operating systems (RTOS) with embedded AI, exploring methodologies for autonomous trajectory estimation, intelligent propulsion management, cognitive communications, and utility-based mission management. By synthesizing advancements in formal verification, embedded systems, and control theory, we establish a reference design for the next generation of autonomous launch vehicles.

2. The Computational Substrate: Hardware and Operating Systems

The avionics of an autonomous rocket serve as its central nervous system. Unlike terrestrial data centers where raw throughput is the primary metric, flight computers must balance performance with extreme reliability, radiation tolerance, and strict timing constraints.

2.1 The Move to Multi-Core and Heterogeneous Computing

Traditional spacecraft relied on radiation-hardened (Rad-Hard) single-core processors like the RAD750, which offer high reliability but meager performance (hundreds of MIPS), insufficient for AI workloads. The demand for autonomy is driving the adoption of high-performance multi-core processors (e.g., ARM Cortex-A53/A72, PowerPC) and heterogeneous Systems-on-Chip (SoCs) like the Xilinx Versal ACAP, which combine scalar processing engines with FPGA logic and AI inference accelerators.

However, multi-core processors introduce "interference channels"—contention for shared resources like Last Level Cache (LLC), memory controllers, and I/O buses. In a safety-critical context, this contention can cause the execution time of a critical task (e.g., guidance loop) to fluctuate unpredictably depending on the activity of a non-critical task (e.g., image logging). This phenomenon, detailed in certification guidance such as CAST-32A, necessitates an Operating System (OS) capable of rigorous resource management and isolation.

2.2 Time and Space Partitioning (TSP) Operating Systems

To safely integrate AI into the control loop, the OS must enforce Time and Space Partitioning. This concept, standardized in ARINC 653, divides the processor into isolated "partitions."

- **Spatial Partitioning:** Uses the Memory Management Unit (MMU) to ensure that a task in Partition A cannot read or write to the memory of Partition B. If an AI trajectory optimizer experiences a buffer overflow or segmentation fault, the OS contains the crash within that partition, protecting the kernel and the critical flight control partition.
- **Temporal Partitioning:** The OS scheduler guarantees a fixed time window for each partition. Even if the AI algorithm enters an infinite loop, the OS forcibly context-switches away from it when its time slice expires, ensuring the stability control laws receive their required CPU time.

VxWorks 653: Wind River's VxWorks 653 is a preeminent example of a commercial RTOS designed for this mixed-criticality environment. It supports robust partitioning and allows for "independent build, link, and load." This is crucial for mission management: the AI software module can be updated or replaced without requiring a complete re-verification of the immutable safety-critical flight control software, significantly reducing lifecycle costs and integration risks.

RTEMS and Open Architectures: For missions prioritizing open standards, the Real-Time Executive for Multiprocessor Systems (RTEMS) provides a flight-proven, deterministic foundation. While historically a single-address-space OS, modern implementations on multi-core architectures (like the GR740 or ARM Cortex-R) are incorporating SMP support and can be deployed within paravirtualized environments (e.g., under the XtratuM hypervisor) to achieve TSP capabilities comparable to commercial solutions. RTEMS is widely used in ESA and NASA missions (e.g., MMS, Solar Orbiter) due to its reliability and lack of licensing fees, making it an attractive baseline for large-scale constellations.

2.3 Middleware: The Connective Tissue

Sitting between the OS and the applications is the middleware, which abstracts the complexity of inter-process communication (IPC) and hardware interfaces.

NASA Core Flight System (cFS): cFS is the de facto standard for spacecraft flight software. Its architecture is defined by the "Software Bus," a publish-subscribe mechanism that decouples applications. In an autonomous rocket, the "Navigation" app publishes a state vector message. The "Guidance" app subscribes to this message, unaware of whether the state vector came from a GPS receiver, an Inertial Measurement Unit (IMU), or a complex Visual Odometry AI. This decoupling enables fault tolerance: if the primary Navigation app fails, a backup app can be started by the cFE (Core Flight Executive) to publish to the same message ID, seamlessly taking over without the subscribers needing reconfiguration.

Data Distribution Service (DDS) and Space ROS: As autonomy requirements grow, the need for more sophisticated data handling arises. Data Distribution Service (DDS) offers a data-centric connectivity framework with granular Quality of Service (QoS) policies (e.g., deadlines, durability, reliability) that are essential for real-time systems. DDS serves as the backbone for ROS 2 (Robot Operating System), which is being adapted for space as "Space ROS". Space ROS hardens the ROS 2 framework for compliance with aerospace standards, enabling the reuse of vast libraries of terrestrial robotics software (e.g., SLAM, motion planning) on spacecraft. The integration of cFS and Space ROS—often using a bridge to translate cFS Software Bus messages to DDS topics—creates a powerful hybrid architecture where the robust, safety-critical functions reside in cFS, while the high-level cognitive and perception tasks run in Space ROS.

Feature	VxWorks 653 (Commercial)	RTEMS (Open Source)	NASA cFS (Middleware)	Space ROS (Framework)
Primary Role	Partitioning Kernel / RTOS	Real-Time Executive / RTOS	Flight Software Framework	Robotics/Autonomy Framework
Criticality Support	Mixed-Criticality (DO-178C)	High Criticality (SMP qualified)	Safety Critical	Performance/Autonomy Focus
Fault Isolation	ARINC 653 (Time/Space)	Library-based (Single Space)	App Management / Heartbeat	Process Isolation (DDS)
Inter-Comms	Partition Ports	Message Queues	Software Bus (Pub/Sub)	DDS (Pub/Sub)
Use Case	Avionics Backbone	Instrument/Subsystem Control	Command & Data Handling	High-Level Planning/Vision

3. Autonomous Trajectory Estimation and Optimization

In the E4 autonomy paradigm, the launch vehicle must generate its own reference trajectory in real-time, adapting to dispersions that would otherwise violate constraints. This moves beyond "Guidance" (following a path) to "Path Planning" (creating the path).

3.1 Convex Optimization: The G-FOLD Breakthrough

The problem of landing a rocket with minimum fuel consumption while respecting thrust limits, glide slope constraints, and pointing requirements is mathematically non-convex (due to the minimum thrust constraint and rotational dynamics). Traditionally, this prevented onboard optimization, forcing reliance on pre-computed look-up tables.
A transformative advancement has been "Lossless Convexification," pioneered by JPL and utilized in algorithms like G-FOLD (Guidance for Fuel-Optimal Large Divert). By relaxing non-convex constraints into a convex Second-Order Cone Programming (SOCP) problem, G-FOLD ensures that a global optimum can be found in deterministic polynomial time. This mathematical guarantee is crucial for flight certification. It allows the onboard computer to re-solve the entire trajectory problem every guidance cycle (e.g., 10-20 Hz), instantly adjusting the flight path if a gust of wind pushes the vehicle off course or if the propellant budget changes.

3.2 Reinforcement Learning for Adaptive Control

While convex optimization excels at "known" physics, Reinforcement Learning (RL) is emerging as a solution for complex, hard-to-model dynamics (e.g., slosh-induced instability or aerodynamic buffeting during trans-sonic flight).
Deep Reinforcement Learning (DRL): Algorithms like Proximal Policy Optimization (PPO) allow an agent to learn a control policy by interacting with a high-fidelity simulator. Unlike PID controllers which are tuned for linear operating points, RL agents can learn non-linear strategies, such as using body lift to extend range or executing "hoverslam" maneuvers that maximize efficiency. To handle the reality gap (Sim-to-Real), techniques like Domain Randomization and Meta-RL are used, training the agent across a distribution of physical parameters (mass, friction, thrust) so that it learns a robust policy adaptable to the actual flight conditions.

Transformer-Based Architectures: Recent research has applied Transformer architectures (Decision Transformers) to the GNC problem. By treating the sequence of states, actions, and rewards as a sequence modeling problem, these models can effectively "attend" to past anomalies. For instance, if a thruster underperforms at t=10s, the Transformer incorporates this context into its decision at t=15s, effectively learning an adaptive control law that compensates for actuator degradation without explicit fault isolation logic.

4. Intelligent Propulsion and Dynamic Fuel Management

The propulsion system is the heart of a launch vehicle, and its management is critical for survival. An autonomous rocket must actively manage its "energy state" and structural integrity through intelligent actuation.

4.1 Fault-Tolerant Control Allocation (FTCA)

In modern multi-engine architectures (e.g., SpaceX Super Heavy, Rocket Lab Electron), the vehicle is over-actuated. The Control Allocation (CA) module is responsible for mapping the virtual force and torque commands from the Guidance system to the physical actuators (engines, grid fins, RCS).

Handling Engine-Out Scenarios: If the Fault Detection, Isolation, and Recovery (FDIR) system identifies an engine failure (e.g., a "Loss of Effectiveness" or a "Lock-in-Place" gimbal failure), the FTCA algorithms must instantly reconfigure. This is not a simple look-up table lookup; it involves solving a constrained optimization problem (often using Weighted Least Squares or Quadratic Programming) to find a new combination of thrust and gimbal angles that satisfies the guidance command.

- **Example:** If an engine on the periphery fails, the FTCA might autonomously throttle down the opposing engine to zero to neutralize the disturbance torque, while throttling up the remaining engines to maintain the total thrust-to-weight ratio > 1. This "virtual reconfiguration" is transparent to the guidance loop, which continues to fly the vehicle as if it were fully functional, albeit with reduced authority.

4.2 Real-Time Fuel Routing and Anomaly Updates

Propellant management is more than just opening valves; it is about maintaining the Center of Mass (CoM) and managing fluid dynamics.

Active CoM Balancing: As fuel is consumed, the CoM shifts, potentially moving outside the controllable envelope of the gimbal system. An AI-driven fuel manager monitors tank levels and flow rates. If it detects an asymmetric drain (e.g., due to a clogged line or leak), it autonomously actuates cross-feed valves to rebalance the load. This logic must be integrated with the GNC system; if the CoM cannot be restored, the Fuel Manager updates the GNC plant model with the new mass properties, allowing the autopilot to adjust its gains to prevent instability.

Anomaly Detection via Transformers: Beyond standard limit-checking, Transformer-based anomaly detection (T-BAD) models analyze the time-series data from turbopumps and injectors. By learning the temporal correlations of nominal operation, these models can detect subtle "precursors" to failure—such as micro-vibrations or slight pressure oscillations—that precede a catastrophic hard start or turbine disintegration. This allows the system to preemptively shut

down a degrading engine before it explodes, preserving the vehicle for a safe abort or degraded mission continuation.

5. Fault Tolerance and Resilience Architectures

The defining characteristic of this architecture is "resilience"—the ability to endure faults and continue the mission.

5.1 The Nervous System: Time-Triggered Ethernet (TTEthernet)

The communication backbone must guarantee that critical data (e.g., IMU rates) arrives on time, every time, regardless of network load. TTEthernet (SAE AS6802) enables this by combining three traffic classes on one wire:

1. **Time-Triggered (TT):** Synchronized to microseconds, used for flight control. These frames have absolute priority.
2. **Rate-Constrained (RC):** Guaranteed bandwidth with bounded latency, used for sensor telemetry (AFDX compliant).
3. **Best-Effort (BE):** Standard Ethernet, used for lower-priority data like video logs or AI diagnostics.

Network-Level Fault Tolerance: TTEthernet switches act as "guardians," filtering out faulty messages (e.g., "babbling idiot" failures where a node floods the network). The topology typically employs a dual or triplex redundant star or ring. If a switch or cable fails, the time-triggered schedule ensures that the redundant copy of the message traversing the parallel path arrives at the destination with *zero* switchover delay, preserving the stability of the control loop. This deterministic behavior allows the "Safety Core" to operate undisturbed even if the "AI Core" saturates the network with Best-Effort traffic.

5.2 The Simplex Architecture: A Safety Net for AI

To safely deploy non-deterministic AI controllers, we employ the **Simplex Architecture**. This architectural pattern wraps the unverified "High-Performance Controller" (HPC - the AI) with a verified "Safety Controller" (SC) and a "Decision Module".

- **Mechanism:** The AI proposes a control action (e.g., "Bank left 30 degrees to optimize landing footprint"). The Decision Module simulates this action forward in time using a simplified physical model to calculate the vehicle's "Forward Reachable Set."
- **Protection:** If the simulation shows that the AI's action would drive the vehicle into an unrecoverable state (e.g., exceeding dynamic pressure limits or geometric commit criteria), the Decision Module *vetoes* the AI command and switches control to the Safety Controller. The SC then executes a proven, conservative maneuver (e.g., "Wings level, max thrust") to stabilize the vehicle.
- **Benefit:** This allows the rocket to utilize aggressive AI optimization for 99% of the flight while maintaining a mathematical guarantee that it will never violate safety boundaries.

5.3 Software-Based Radiation Tolerance

Modern AI requires the computational density of GPUs or advanced SoCs (like the Snapdragon or Versal), which are intrinsically soft to radiation (Single Event Upsets - SEUs). To use them,

we employ Software Implemented Hardware Fault Tolerance (SIHFT).
Triple Modular Redundancy (TMR) in Software: Critical processes are executed in parallel on three different cores of a multi-core processor. A software voter compares the outputs. If a radiation particle flips a bit in Core 2, the voter detects that its output differs from Core 1 and Core 3, discards the result, and rewrites the correct state to Core 2's memory to "scrub" the error. This allows COTS hardware to achieve reliability levels approaching that of rad-hard components.

6. Cognitive Communications and Network Restoration

An autonomous rocket is a node in a celestial network. It must maintain connectivity with ground stations, relay satellites (TDRSS, Starlink), and potentially other spacecraft.

6.1 Cognitive Radio and Autonomous Link Recovery

Traditional radios rely on fixed frequencies and schedules. A Cognitive Radio (CR) embedded with AI can sense the electromagnetic spectrum to detect interference or jamming.
Policy-Based Reconfiguration: The CR engine uses a policy database to optimize link parameters. If the primary X-band link degrades due to rain fade or jamming, the AI automatically reconfigures the Software Defined Radio (SDR) to switch to Ka-band or an optical link, adjusting modulation schemes (e.g., dropping from QAM-64 to QPSK) to maintain link budget.
Automated Reconnection Logic: In a "Lost Link" scenario, the avionics executes a hierarchical recovery procedure:

1. **Sidelobe Search:** The system commands the phased array antennas to electronically steer beams in a search pattern.
2. **Attitude Recovery:** If electronic steering fails, the vehicle physically reorients (slews) to point high-gain antennas toward the last known Earth ephemeris or alternative relay satellites.
3. **Protocol Handshake:** Once a carrier is detected, the radio autonomously negotiates session keys and bit rates, prioritizing critical command/telemetry channels over bulk science data.

6.2 Disruption Tolerant Networking (DTN)

In deep space or tumbling scenarios, continuous connectivity is impossible. Standard TCP/IP fails here because it requires an end-to-end handshake.
Custody Transfer: The architecture utilizes the Bundle Protocol (BP). When the rocket generates data, it is encapsulated in a "bundle." If the link is down, the bundle is stored in non-volatile memory. When a link becomes available (e.g., a momentary pass of a relay satellite), the bundle is transmitted. Crucially, the rocket does not delete the data until it receives a "Custody Acceptance" signal from the next hop. This "store-and-forward" mechanism ensures data is never lost, even if the path to Earth is broken for hours.
Self-Healing Mesh Topologies: In multi-spacecraft missions (e.g., a swarm deployment), the vehicles form a mesh network. Software Defined Networking (SDN) controllers on the "mother" ship or ground dynamically calculate routing tables. If Node A fails, the SDN controller updates

the flow rules to route packets through Node B, healing the network partition autonomously.

7. Mission Management: The Utility-Based Brain

At the top of the hierarchy sits the Mission Manager, responsible for "what to do" rather than "how to do it."

7.1 Goal-Oriented Autonomous Controller (GOAC)

Instead of time-tagged command sequences, the rocket is given high-level goals (e.g., "Insert Payload X into Orbit Y"). The GOAC architecture uses a Planner (such as a Hierarchical Task Network - HTN planner) to decompose this goal into primitive actions (e.g., "Open Valves," "Ignite Engine," "Monitor Thrust").
Reactive Execution: The Executive layer (e.g., T-REX or PLEXIL) dispatches these actions. Unlike a linear sequencer, the Executive monitors the *outcome*. If "Ignite Engine" fails to produce pressure, the Executive does not blindly proceed; it catches the failure, queries the FDIR system, and requests a new plan from the Planner (e.g., "Attempt restart" or "Switch to backup engine"). This closed-loop task execution is fundamental to resilience.

7.2 Utility Accrual Scheduling (UAS)

A key feature of intelligent autonomy is the ability to handle resource conflicts (e.g., low battery vs. science opportunity). UAS assigns a mathematical "Utility Function" to every task—a curve representing the value of completing the task at a specific time.
Priority Modules: When resources are constrained, the Scheduler calculates the "Utility Density" (Value per unit of Resource) for all candidate tasks. It constructs a schedule that maximizes the total accrued utility.

- *Scenario:* A solar flare event is detected. The "Space Weather Monitor" task has a step-function utility (extremely high value if done *now*). The "Downlink Science Data" task has a linear utility. The Scheduler autonomously pauses the downlink to power the radiation sensors and close the radiation shutters, realizing that preserving the vehicle has infinite utility compared to the latency of science data. This prioritization happens dynamically, without ground intervention.

8. Environmental Analysis and Situational Awareness

To operate autonomously, the rocket must perceive and interpret its physical reality.

8.1 Auto-Analyzing Space/Ground Environments

Ground Analysis (ALHAT): During landing, the vehicle uses "Autonomous Landing and Hazard Avoidance Technology" (ALHAT). A Flash Lidar scans the surface, generating a 3D Digital Elevation Model (DEM). A Convolutional Neural Network (CNN) processes this map to identify hazards (rocks, slopes > 5°). The system then identifies the reachable "safe harbor" landing sites, scores them based on fuel cost and safety margins, and retargets the guidance system to the optimal spot.
Space Analysis (Space Weather & Debris): In orbit, the vehicle uses optical cameras and

radiation sensors.

- *Space Weather:* Onboard AI models (LSTMs) analyze particle flux rates to predict Solar Particle Events (SPEs). If a storm is predicted, the system autonomously enters "Safe Mode," powering down sensitive non-essential electronics and reorienting the water/fuel tanks to shield the flight computer.
- *Debris Avoidance:* Optical sensors track objects moving relative to the background starfield. If a collision trajectory is detected (using ELVO algorithms), the autonomy system calculates a "Collision Avoidance Maneuver," checks fuel constraints, and executes the burn.

9. Conclusion

The architecture defined in this report represents a convergence of robust systems engineering and cutting-edge artificial intelligence. By layering a Goal-Oriented Mission Manager atop a Fault-Tolerant OS and connecting them via a Cognitive Network, we create a launch vehicle that is not just a delivery system, but a resilient, self-preserving robot. The use of Simplex Architectures and TTEthernet bridges the gap between the stochastic nature of AI and the deterministic requirements of flight safety, enabling the "best of both worlds."
As humanity pushes further into the cosmos, the tether to Earth must inevitably loosen. This architecture provides the technological foundation for that independence, ensuring that future rockets can think, adapt, and survive in the harsh and unforgiving environment of space.

Subsystem	Conventional Approach (E1)	Autonomous Approach (E4)	Key Technologies
OS/Kernel	Single Criticality, Federated	Mixed-Criticality, Partitioned (TSP)	VxWorks 653, RTEMS-SMP, Hypervisors
Guidance	Ground-computed, Table-lookup	Onboard Optimization, Adaptive	Convex Optimization (G-FOLD), RL, Transformers
Propulsion	Fixed mixing logic, Abort on fail	Dynamic Control Allocation, Reconfiguration	FTCA, Matrix Pseudoinverse, Optimization
Comms	Continuous link, Dumb pipe	Store-and-Forward, Cognitive	DTN (Bundle Protocol), Cognitive Radio, SDN
Planning	Time-tagged sequences	Goal-oriented, Utility-based	HTN Planners, Utility Accrual Scheduling
Safety	Hardware Voting, Ground intervention	Runtime Assurance, Simplex	Simplex Architecture, Reachability Analysis

Works cited

1. Autonomy Requirements Engineering - Indico at ESA / ESTEC, https://indico.esa.int/event/42/contributions/2442/attachments/1964/2273/Autonomy_Requirements_Engineering.pdf 2. Taking the ECSS Autonomy Concepts One Step Further, https://www.inpe.br/noticias/arquivos/pdf/SpaceOps2010.pdf 3. A Goal-Oriented Autonomous Controller for Space Exploration | Istituto di Scienze e Tecnologie della Cognizione - Istc-Cnr, https://www.istc.cnr.it/en/content/goal-oriented-autonomous-controller-space-exploration 4. Enhancing Fault-Tolerant Space Computing: Guidance Navigation and Control (GNC) and

Landing Vision System (LVS) Implementations - arXiv, https://arxiv.org/pdf/2511.04052 5. Spacecraft Autonomy Challenges for Next-Generation Space Missions - Autonomous Systems Laboratory, https://stanfordasl.github.io/wp-content/papercite-data/pdf/Starek.Acikmese.ea.ACSTAA16.pdf 6. Project Morpheus - Wikipedia, https://en.wikipedia.org/wiki/Project_Morpheus 7. Project Morpheus: Lessons Learned in Lander Technology Development, https://ntrs.nasa.gov/api/citations/20140001410/downloads/20140001410.pdf 8. Toward Runtime Assurance of Complex Systems with AI Components - PHM Society, https://papers.phmsociety.org/index.php/phme/article/download/3361/1920 9. Architecting Safer Autonomous Aviation Systems† - NASA Technical Reports Server, https://ntrs.nasa.gov/api/citations/20220016762/downloads/fnpw-SSS2023-final.pdf 10. VxWorks 653 Multi-core Edition Product Overview - Wind River, https://www.windriver.com/resource/vxworks-653-product-overview 11. VxWorks 653: ARINC 653–Compliant Platform for IMA Marketplace - Mistral Solutions, https://test1.mistralsolutions.com/defence_products/wind-river-vxworks-653/ 12. Time-Triggered Ethernet - TTTech Computertechnik AG, https://www.tttech.com/know-how/time-triggered-ethernet 13. Towards Autonomous Time-Triggered Ethernet - Flight Software Workshop, https://flightsoftware.jhuapl.edu/files/2010/FSW10_Steiner.pdf 14. ROS and cFS System (RACS): Easing Space Robotic Development - IEEE Xplore, https://ieeexplore.ieee.org/iel7/9438108/9438128/09438288.pdf 15. Space ROS: An Open-Source Framework for Space Robotics and Flight Software - NASA Technical Reports Server, https://ntrs.nasa.gov/api/citations/20220017761/downloads/Space_ROS_SciTech.pdf 16. AMD Versal™ XQR Adaptive SoCs, https://www.amd.com/en/products/adaptive-socs-and-fpgas/versal/space-grade.html 17. XQR Versal™ for Space 2.0 Applications - AMD, https://www.xilinx.com/content/dam/xilinx/publications/product-briefs/xilinx-xqr-versal-product-brief.pdf 18. Mixing Modern Multi-core Processors with Open Architectures - Wind River, https://www.windriver.com/sites/default/files/2024-04/603063_Whitepaper-ScioTeq-CoreAVI-WindRiver.pdf 19. Mixing Modern Multi-core Processors with Open Architectures | Wind River, https://www.windriver.com/resource/mixing-modern-multi-core-processors-with-open-architectures 20. RTEMS in the Space Domain - embedded brains, https://embedded-brains.de/en/rtems-en/rtems-in-the-space-domain/ 21. Embedded systems for space exploration and satellite communication - World Journal of Advanced Research and Reviews, https://wjarr.com/sites/default/files/WJARR-2021-0341.pdf 22. MMS Instruments Suite - Applications, https://www.rtems.org/applications/hardware/mms_instruments_suite/ 23. core Flight System (cFS) - ETD | Goddard Engineering and Technology Directorate - NASA, https://etd.gsfc.nasa.gov/capabilities/capabilities-listing/cfs/ 24. core Flight System - ETD | Goddard Engineering and Technology Directorate - NASA, https://etd.gsfc.nasa.gov/capabilities/core-flight-system/ 25. Core Flight System (cFS)Training - NASA Technical Reports Server, https://ntrs.nasa.gov/api/citations/20205011588/downloads/TM%2020205000691%20REV%201.pdf 26. Aerospace & Defense (A&D) | Mission-Critical Systems - RTI, https://www.rti.com/industries/aerospace-defense 27. Data Distribution Service (DDS) - Object Management Group, https://www.omg.org/omg-dds-portal/ 28. FAQ - Space ROS, https://space.ros.org/pages/faq.html 29. [1802.03827] Successive Convexification for 6-DoF Mars Rocket Powered Landing with Free-Final-Time - arXiv, https://arxiv.org/abs/1802.03827 30. Optimal Rocket Landing Guidance Using Convex Optimization and Model Predictive Control

- Aerospace Research Central, https://arc.aiaa.org/doi/10.2514/1.G003518 31. How Does SpaceX Optimize Rocket Launches? A Convex Optimization Playground - Reddit, https://www.reddit.com/r/SpaceXLounge/comments/1gcs4kz/how_does_spacex_optimize_rocket_launches_a_convex/ 32. Space Trajectory Planning with a General Reinforcement-Learning Algorithm - MDPI, https://www.mdpi.com/2226-4310/12/4/352 33. Rocket Trajectory Optimization with Reinforcement Learning - Medium, https://medium.com/@arnobgr/rocket-trajectory-optimization-with-reinforcement-learning-6637f5af21bf 34. 6-DOF Atmospheric Rocket Landing Guidance using Meta-Reinforcement Learning - electronic library -, https://elib.dlr.de/210766/1/Jacopo_Carradori_Thesis_reduced.pdf 35. Multi-Phase Spacecraft Trajectory Optimization via Transformer-Based Reinforcement Learning - arXiv, https://arxiv.org/html/2511.11402v1 36. Transformer-Based Anomaly Detection in Deep Reinforcement Learning - ResearchGate, https://www.researchgate.net/publication/397444211_Transformer-Based_Anomaly_Detection_in_Deep_Reinforcement_Learning 37. Machine Learning Based Fault Tolerant Thrust Vector Control of a Small Launch Vehicle - Middle East Technical University, https://open.metu.edu.tr/bitstream/handle/11511/113066/Machine%20Learning%20Based%20Fault%20Tolerant%20Thrust%20Vector%20Control%20of%20a%20Small%20Launch%20Vehicle.pdf 38. Optimal reconfigurable allocation of a multi-engine cluster for a reusable launch vehicle - eucass, https://www.eucass.eu/component/docindexer/?task=download&id=6683 39. Multiple model fault diagnosis and fault tolerant control for the launch vehicle's attitude control system | The Aeronautical Journal | Cambridge Core, https://resolve.cambridge.org/core/journals/aeronautical-journal/article/multiple-model-fault-diagnosis-and-fault-tolerant-control-for-the-launch-vehicles-attitude-control-system/3A4F027C43344C44435440ED111FDA80/core-reader 40. Fault-Tolerant Control for a Cluster of Rocket Engines - Results for launch and landing of a re-usable launcher - CEAS, https://eurognc.ceas.org/archive/EuroGNC2024/pdf/CEAS-GNC-2024-045.pdf 41. SpaceX Acknowledges Falcon 9 Engine Anomaly - NASA Spaceflight Forum, https://forum.nasaspaceflight.com/index.php?topic=26739.80 42. Transformer-Based Anomaly Detection in Deep Reinforcement Learning - Article Details, http://poster-openaccess.com/article_detail.php?paper_id=442&conf=ICIC&year=2025 43. Towards Comprehensive AI-Based Onboard FDIR: System Design and First Results - DigitalCommons@USU, https://digitalcommons.usu.edu/cgi/viewcontent.cgi?article=5708&context=smallsat 44. Motor Controllers in Space | VORAGO Technologies, https://www.voragotech.com/blog/deterministic-ethernet-space-applications 45. Highly reliable TTEthernet® controllers from TTTech Aerospace improve Ariane 6 avionics system, https://www.tttech.com/press/tttech-ttethernet-controllers-ariane-6-avionics 46. A Verification Framework for Runtime Assurance of Autonomous UAS - NASA Technical Reports Server (NTRS), https://ntrs.nasa.gov/citations/20240007986 47. The Black-Box Simplex Architecture for Runtime Assurance of Autonomous CPS, https://par.nsf.gov/servlets/purl/10327769 48. Run Time Assurance for Spacecraft Attitude Control Under Nondeterministic Assumptions - Sam Coogan, https://coogan.ece.gatech.edu/papers/pdf/abate2022runtime.pdf 49. The Use of the Simplex Architecture to Enhance Safety in Deep-Learning-Powered Autonomous Systems - arXiv, https://arxiv.org/html/2509.21014v1 50. [2302.02885] Run Time Assurance for Autonomous Spacecraft Inspection - arXiv, https://arxiv.org/abs/2302.02885 51. Current Research: - Radiation Tolerance - PDOS-MIT, https://pdos.csail.mit.edu/6.828/2021/lec/CelSkeggsRadiationTolerance.pdf 52. Single Event Upset Mitigation Techniques for SRAM-based FPGAs - Instituto de Informática - UFRGS,

https://www.inf.ufrgs.br/~fglima/lima_latw03.pdf 53. Radiobots: Autonomous Cognitive Radios for Jammer Resistant SATCOM and SIGINT 2013 - Office of Research & Innovation - Naval Postgraduate School, https://nps.edu/web/research/w/radiobots-autonomous-cognitive-radios-for-jammer-resistant-satcom-and-sigint-2013 54. Cognitive Engine One: A Cross-Layer Framework for Autonomy in Multi-Provider Space Communications Environments, https://ntrs.nasa.gov/api/citations/20240012466/downloads/ce1_preprint_1017.pdf 55. "Satellite Reorientation Using Reinforcement Learning Under Unknown Att" by Matthew Willoughby - Scholarly Commons, https://commons.erau.edu/edt/901/ 56. Deep Learning Frameworks for Cognitive Radio Networks: Review and Open Research Challenges - arXiv, https://arxiv.org/html/2410.23949v1 57. An Overview of Cognitive Radio Technology and Its Applications in Civil Aviation - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC10346798/ 58. Delay-tolerant networking - Wikipedia, https://en.wikipedia.org/wiki/Delay-tolerant_networking 59. Disruption Tolerant Networking - SciSpace, https://scispace.com/pdf/disruption-tolerant-networking-3v0jcil47n.pdf 60. RFC 4838: Delay-Tolerant Networking Architecture, https://www.rfc-editor.org/rfc/rfc4838.html 61. A Mesh Network for Joint Indo-Pacific Operations - Booz Allen, https://www.boozallen.com/markets/defense/indo-pacific/a-mesh-network-for-joint-indo-pacific-operations.html 62. Software-defined satellite network: a survey | by Andrey - Medium, https://medium.com/@andrey012340/software-defined-satellite-network-a-survey-ecb71c378fb8 63. LEO SOFTWARE DEFINED NETWORKING BASED ON ONBOARD CONTROLLER, https://ieeexplore.ieee.org/document/8646688/ 64. (PDF) A Goal-Oriented Autonomous Controller for space exploration - ResearchGate, https://www.researchgate.net/publication/228784908_A_Goal-Oriented_Autonomous_Controller_for_space_exploration 65. executive_trex - ROS Wiki, https://wiki.ros.org/executive_trex 66. A Deliberative Architecture for AUV Control, https://ros.fei.edu.br/roswiki/attachments/trex_executive/TREX.ICRA.08.pdf 67. A Utility Accrual Scheduling Algorithm for Real-Time Activities With Mutual Exclusion Resource Constraints - DTIC, https://apps.dtic.mil/sti/tr/pdf/ADA456339.pdf 68. A utility accrual scheduling algorithm for real-time activities with mutual exclusion resource constraints - ResearchGate, https://www.researchgate.net/publication/3045030_A_utility_accrual_scheduling_algorithm_for_real-time_activities_with_mutual_exclusion_resource_constraints 69. Improved Algorithms for Co-Scheduling of Edge Analytics and Routes for UAV Fleet Missions, https://par.nsf.gov/servlets/purl/10634042 70. A Deep learning approach to Hazard detection for Autonomous Lunar landing - The University of Texas at Austin, https://sites.utexas.edu/near/files/2020/11/CNN_landing_accepted.pdf 71. ALHAT: Autonomous Landing and Hazard Avoidance Technology - JPL Robotics - NASA, https://www-robotics.jpl.nasa.gov/what-we-do/research-tasks/alhat-autonomous-landing-and-hazard-avoidance-technology/ 72. [2304.01583] HALO: Hazard-Aware Landing Optimization for Autonomous Systems - arXiv, https://arxiv.org/abs/2304.01583 73. New AI Tool Aims to Protect Military Satellites from Space Weather, https://www.nationaldefensemagazine.org/articles/2025/12/12/new-ai-tool-aims-to-protect-military-satellites-from-space-weather 74. Space AI Leveraging Artificial Intelligence for Space to Improve Life on Earth - arXiv, https://arxiv.org/html/2512.22399v1 75. ELVO-Based Autonomous Satellite Collision Avoidance with Multiple Debris - MDPI, https://www.mdpi.com/2226-4310/12/5/402 76. Mission Planning on Autonomous Avoidance for Spacecraft Confronting Orbital Debris, https://arxiv.org/html/2409.09959v1